FINAL LAB – HEALTH APP

Intro

The Food Macro Tracker is a web application that allows users to browse, search and manage nutritional information for different foods. The application was developed using Node.js, Express and MySQL, with EJS templates used to generate dynamic web pages.

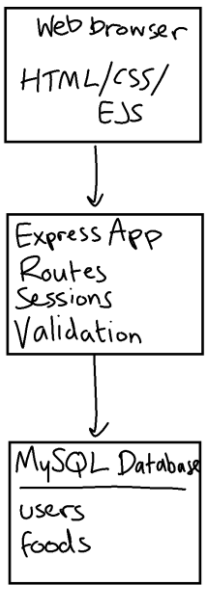
Visitors can view all foods stored in the database and use the search page to find foods by name. Each food displays its calories, protein, carbohydrates and fat values, allowing users to quickly compare nutritional information.

Users who log in using the provided account gain access to additional functionality. Authenticated users can add new foods to the database through a form, edit existing food entries when nutritional information changes, and delete foods that are no longer required. Session-based authentication ensures that only logged-in users can modify the database, while visitors can still browse and search the food list.

The application stores all data within a MySQL database consisting of separate tables for users and foods. SQL queries are parameterised to reduce the risk of SQL injection, and the project includes SQL scripts that recreate the database and populate it with sample data, allowing the application to be installed easily on another computer.

Architecture

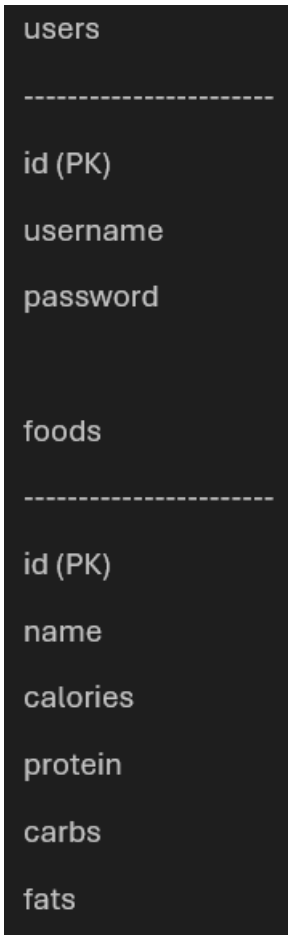
The application follows a three-tier architecture consisting of the presentation layer, application layer and data layer.



The presentation layer uses HTML, CSS and EJS templates to display pages to the user. Requests are processed by an Express server, which handles routing, authentication and communication with the MySQL database. Session middleware stores login information so authenticated users can access protected features. The data layer consists of a MySQL database containing user accounts and food records. Database queries are executed using the mysql2 package with parameterised SQL statements.

Data Model

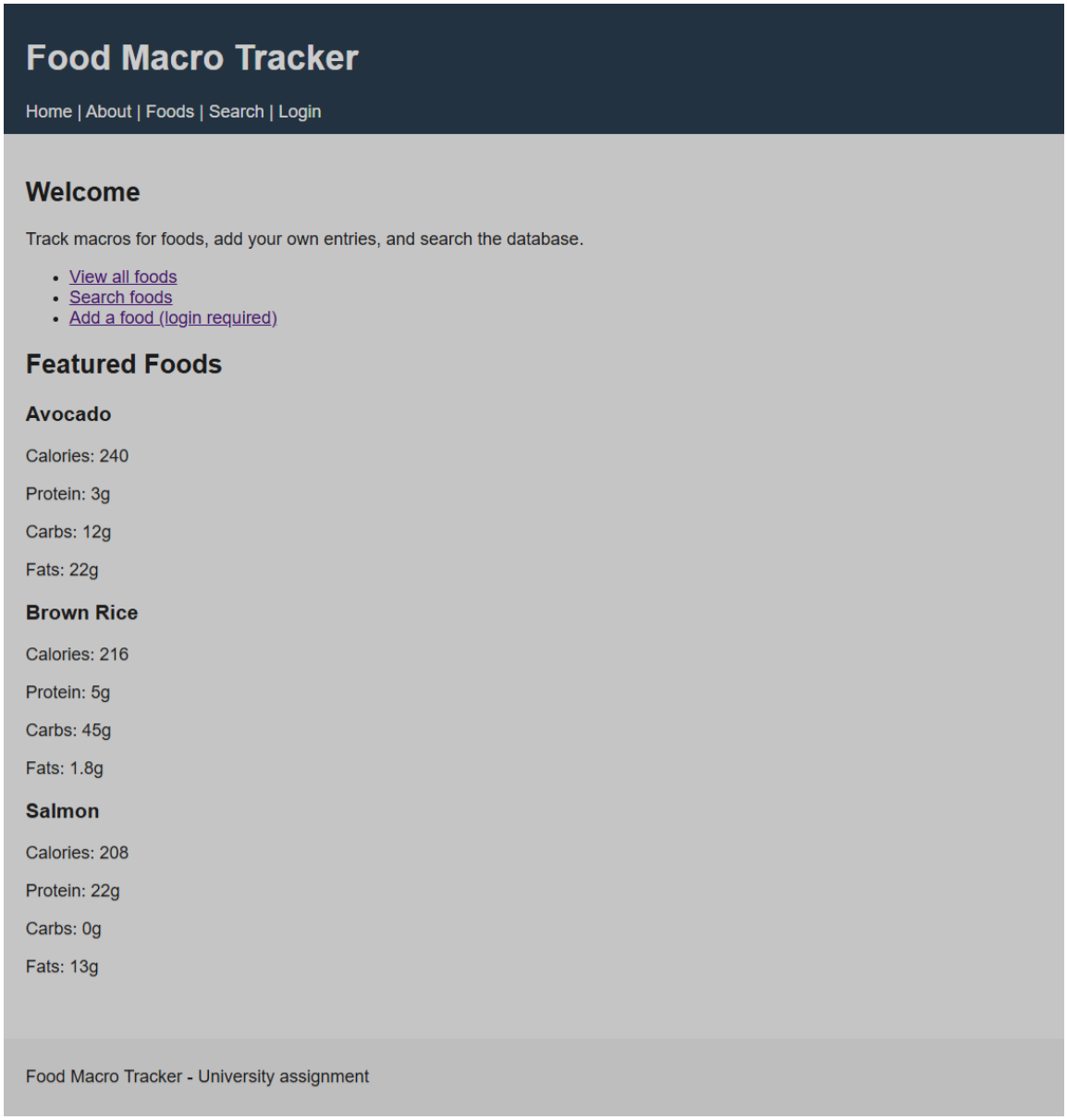
The database consists of two related tables. The users table stores login credentials used for authentication, while the foods table stores nutritional information for each food item, including calories, protein, carbohydrates and fats. Each food is identified by a unique primary key, allowing records to be edited or deleted individually. Separating user accounts from food information improves organisation and allows authenticated users to manage the food database securely.



User Functionality

Home Page

The home page acts as the main entry point to the application and provides users with navigation links to the main features of the website, including viewing foods, searching, logging in and learning more about the application. The page also displays a selection of featured foods from the database, allowing users to immediately view examples of nutritional information without needing to perform a search. This gives users an overview of the application's purpose and encourages interaction with the database.

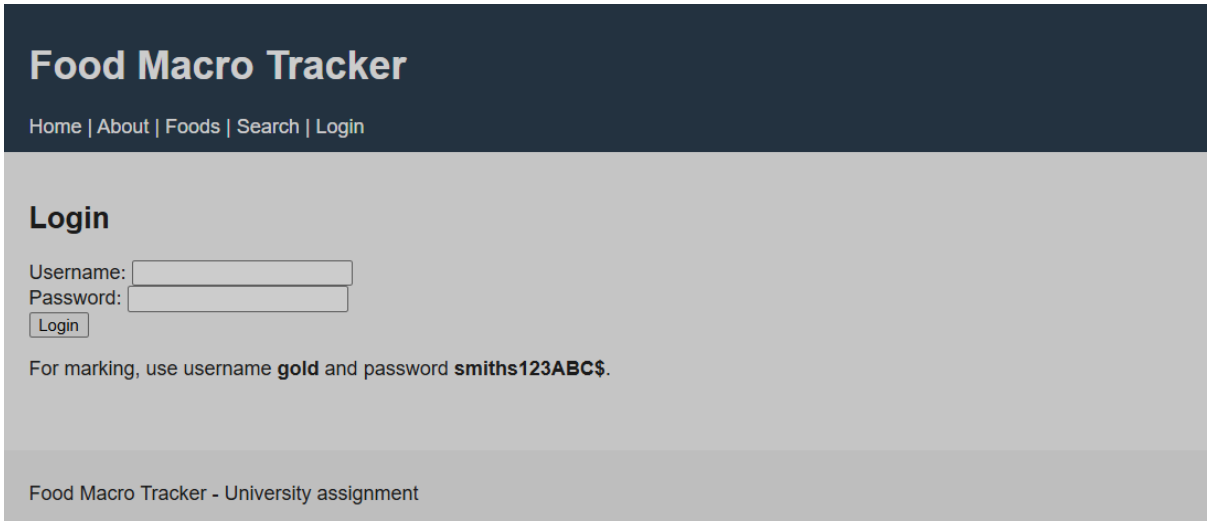


About Page

The About page explains the purpose of the Food Macro Tracker and describes how it can be used to store and view nutritional information. It also outlines the main technologies used during development, including Node.js, Express, MySQL and EJS. This page provides users with background information about the application and explains how the different technologies work together to deliver the website.

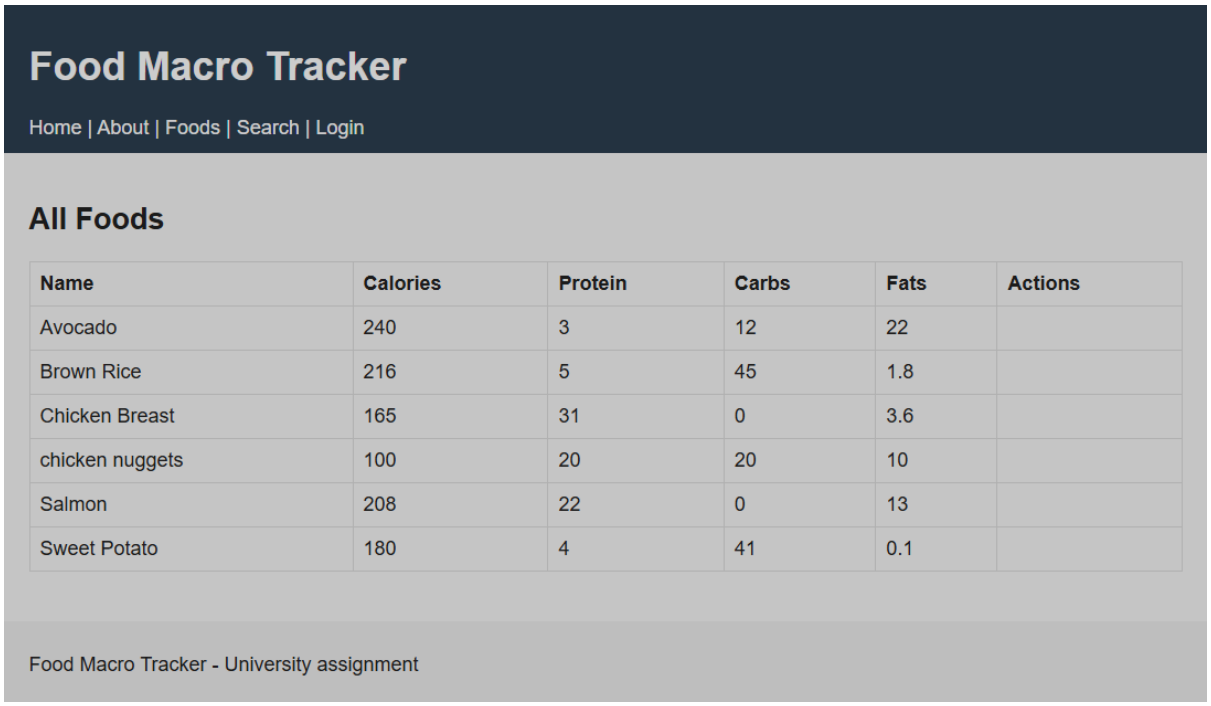
Login

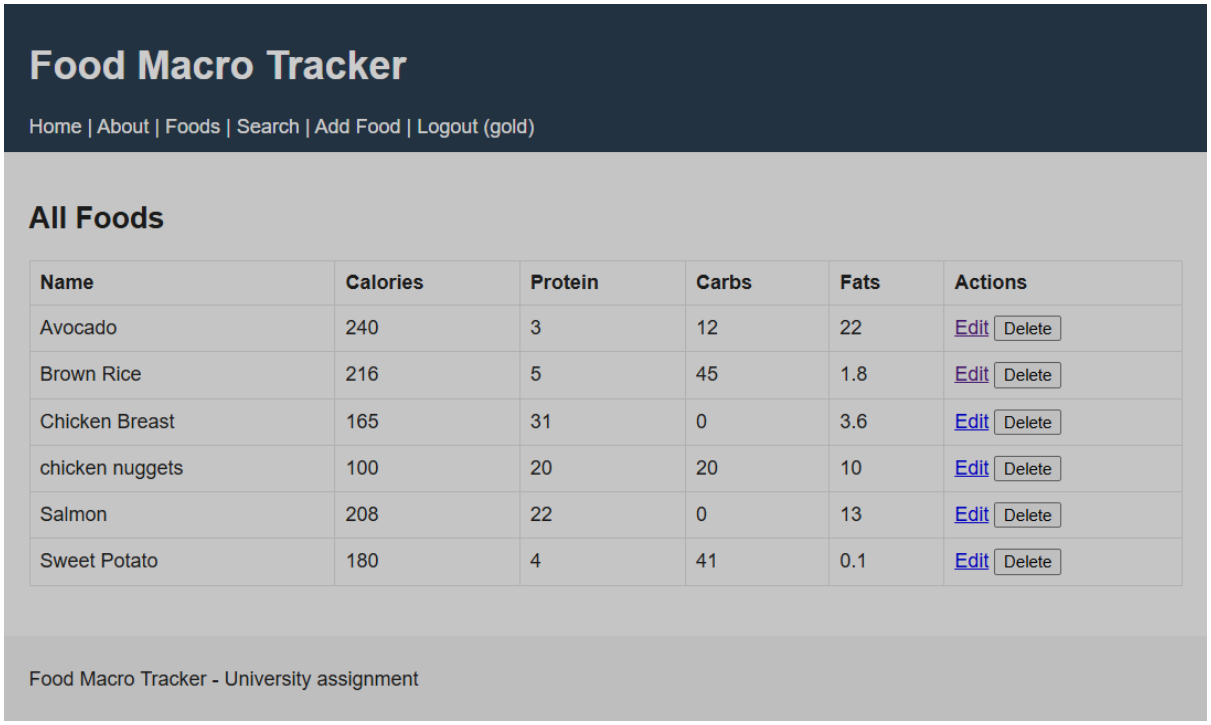
The application uses a login system to restrict database modifications to authenticated users. Users log in using the provided username and password, after which a session is created using the Express Session middleware. Protected routes ensure that only authenticated users can access pages used to add, edit or delete food records. Visitors who are not logged in can still browse and search the database but cannot make any changes.



Food List

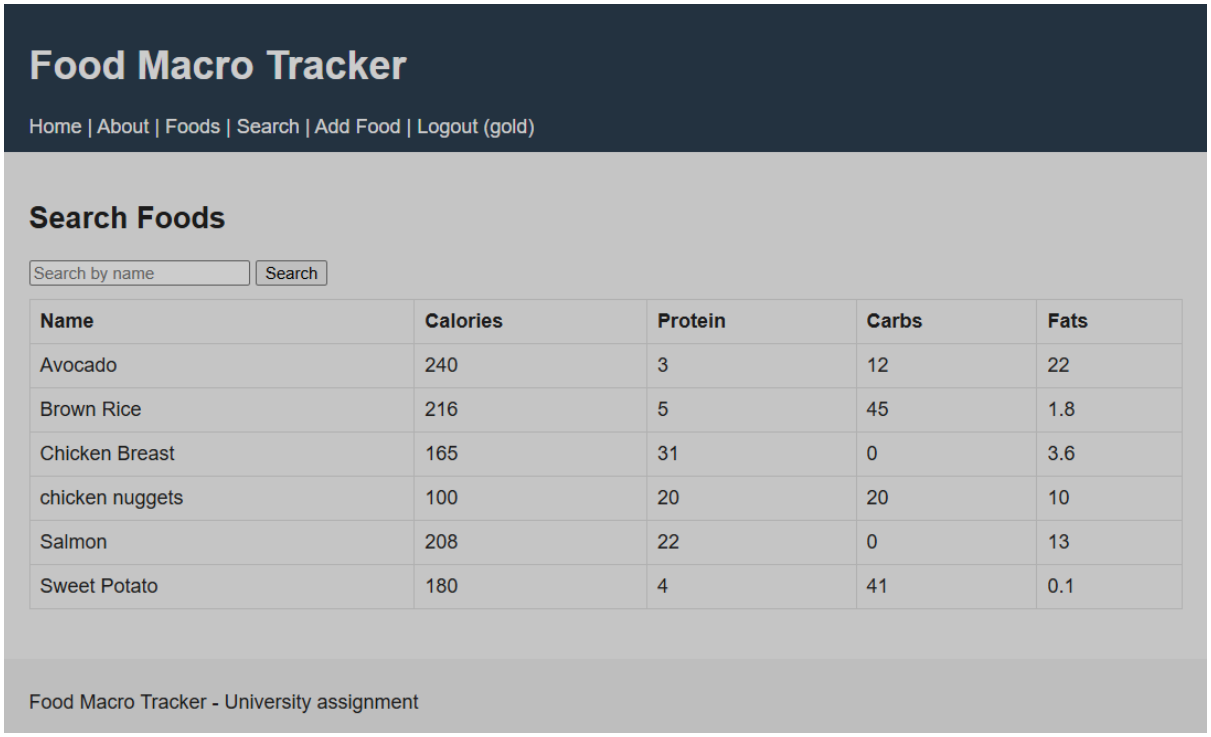
The Food List page retrieves all food records from the MySQL database and displays them in alphabetical order within a table. Each record shows the food's calories, protein, carbohydrates and fat values. When a user is logged in, an additional **Actions** column becomes visible, allowing foods to be edited or deleted directly from the table. Visitors who are not logged in are only able to view the nutritional information.





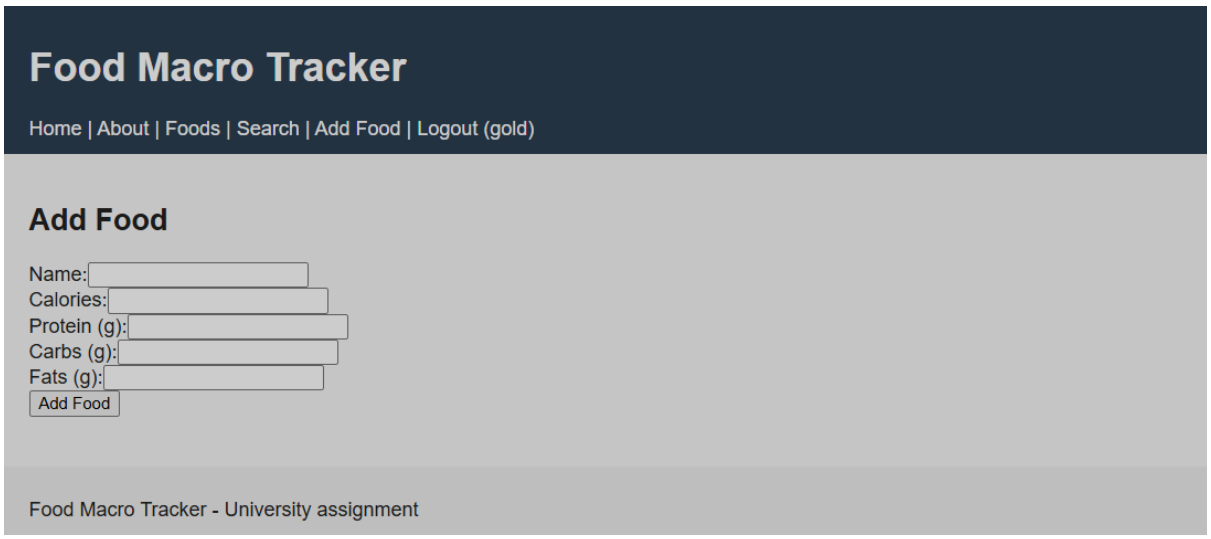
Search

The Search page enables users to search for foods by entering a keyword into a search box. The application performs a parameterised SQL query using the `LIKE` operator to find foods whose names match the search term. Matching results are displayed in a table, allowing users to quickly locate specific food items stored in the database.



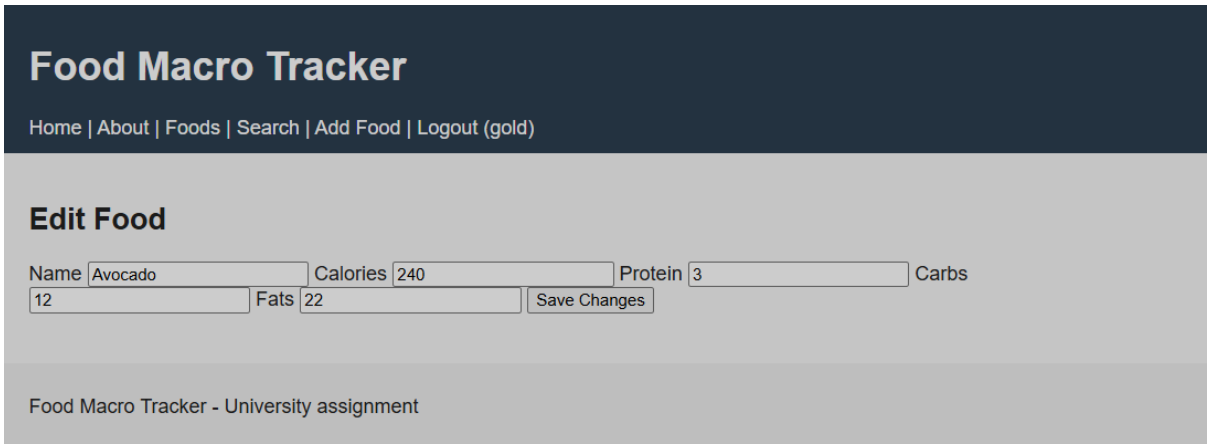
Add Food

Authenticated users can add new foods using a HTML form. Users enter the food name along with its calories, protein, carbohydrates and fat values. The application validates the submitted information before inserting the new record into the MySQL database. Once successfully added, the user is redirected back to the Food List page where the new food is immediately visible.



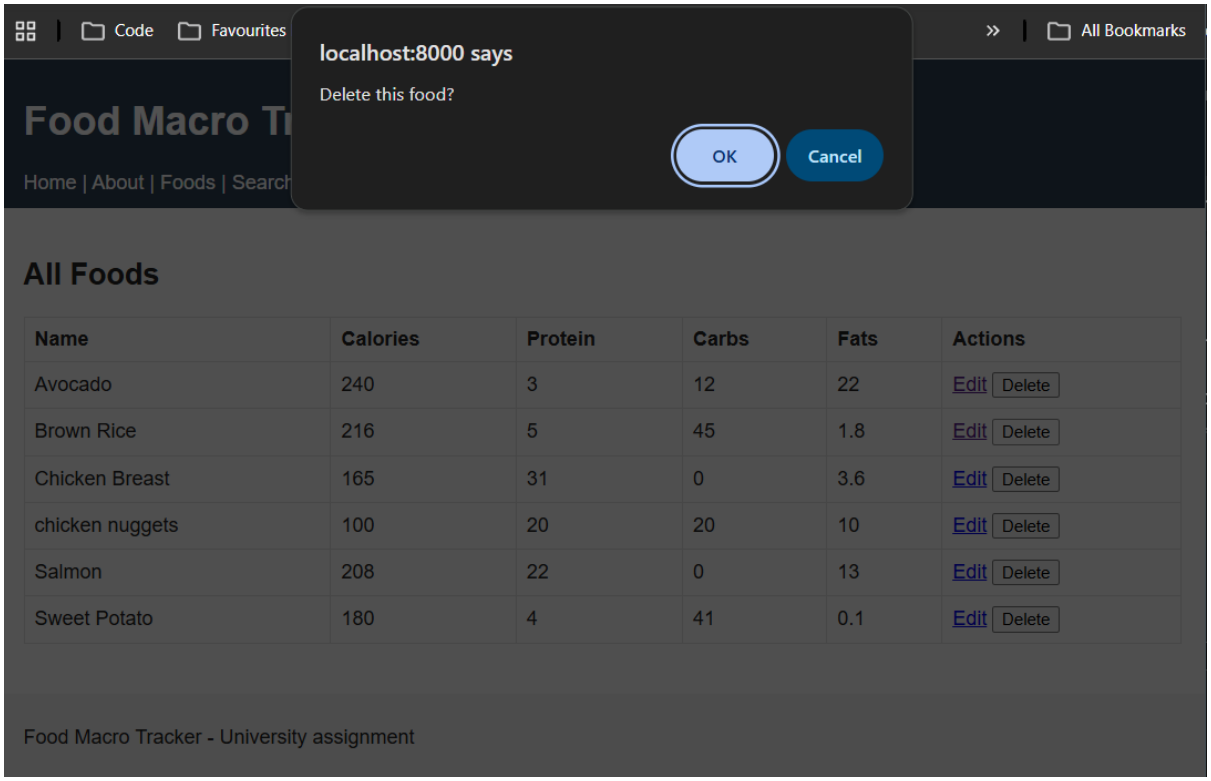
Edit Food

Logged-in users can edit existing food records by selecting the **Edit** option from the Food List page. The edit form is automatically pre-filled with the current nutritional information, allowing users to modify only the values that require updating. When the form is submitted, the application performs an SQL UPDATE query and the revised information is immediately reflected within the food table.



Delete Food

Authenticated users can delete food records directly from the Food List page. Selecting the **Delete** button displays a confirmation prompt to prevent accidental deletion. If confirmed, the application executes an SQL DELETE query to remove the selected record from the database. The Food List page is then refreshed automatically, showing the updated list without the deleted food.



Additional Techniques

Although the application satisfies the compulsory requirements of the assignment, several additional techniques were implemented to improve the security, maintainability and overall usability of the system.

Session Authentication

The application uses the express-session package to manage user authentication. When a user logs in successfully, their username is stored in the session, allowing the application to recognise authenticated users across multiple pages. Protected routes such as Add Food, Edit Food and Delete Food check whether a valid session exists before allowing access. If a user is not logged in, they are redirected to the login page. This prevents unauthorised users from modifying the database while still allowing all visitors to browse and search the food database.

Parameterised SQL Queries

All SQL statements that receive user input use parameterised queries provided by the mysql2 package. Rather than inserting user input directly into SQL strings, placeholders (?) are used and the values are supplied separately. This approach reduces the risk of SQL injection attacks and ensures that user input is handled safely. Parameterised queries are used throughout the application for searching, inserting, updating and deleting food records.

Express Routing

The application separates functionality into multiple Express router files instead of placing all routes inside a single file. This makes the project easier to organise, read and maintain. General pages such as the home and about pages are handled in pages.js, user authentication is managed in auth.js, and all food-related functionality, including searching and CRUD operations, is contained in foods.js. Splitting responsibilities into dedicated routers follows good software engineering practice and makes future development easier.

EJS Templates

The user interface was built using Embedded JavaScript (EJS) templates. EJS allows dynamic content from the database to be displayed directly within HTML pages. Reusable header and footer partials are included on every page, reducing duplicated code and making navigation consistent throughout the application. EJS is also used to conditionally display features depending on whether a user is logged in. For example, the Edit and Delete options only appear to authenticated users, while visitors can only view the nutritional information.

Ai Declaration

AI tools were used to support development by explaining Express, MySQL and EJS concepts, assisting with debugging database connection issues, suggesting improvements to routing and CRUD functionality, and helping draft the accompanying documentation. All code was reviewed, tested and integrated by the author, who was responsible for implementing and verifying the final application.